SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO1: *Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05 **Total Marks:100**

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

I HARI VISHNU N (192524319) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Exploratory Data Analysis-Based Practical Assessment

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

CODE:

```
from google.colab import files

uploaded = files.upload()
import pandas as pd
import matplotlib.pyplot as plt

# Read CSV File
df = pd.read_csv("1students_data.csv")

# Display Dataset
print("========== DATASET ==========")
print(df)

# 1. Number of Rows and Columns
print("\n1. Shape of Dataset")
print(df.shape)
```

```
# 2. Data Types
print("\n2. Data Types")
print(df.dtypes)

# 3. Missing Values
print("\n3. Missing Values")
print(df.isnull().sum())

# Duplicate Values
print("\nDuplicate Values")
print(df.duplicated().sum())

# 4. Statistical Measures

print("\nMean")
print(df.mean(numeric_only=True))

print("\nMedian")
print(df.median(numeric_only=True))

print("\nMinimum")
print(df.min(numeric_only=True))

print("\nMaximum")
print(df.max(numeric_only=True))

print("\nStandard Deviation")
print(df.std(numeric_only=True))

# Correlation Matrix
print("\nCorrelation Matrix")
print(df[["Attendance","Study_Hours","Internal_Marks"]].corr())

# Average values based on Result
print("\nAverage Values based on Result")
print(df.groupby("Result")[["Attendance","Study_Hours","Internal_Marks"]].mean())

# Bar Chart
df["Result"].value_counts().plot(kind="bar")
plt.title("Pass vs Fail")
plt.xlabel("Result")
plt.ylabel("Number of Students")
plt.show()

# Histogram
plt.hist(df["Attendance"], bins=5)
plt.title("Attendance Distribution")
plt.xlabel("Attendance")
plt.ylabel("Frequency")
plt.show()
```

```
# Scatter Plot
plt.scatter(df["Study_Hours"], df["Internal_Marks"])
plt.title("Study Hours vs Internal Marks")
plt.xlabel("Study Hours")
plt.ylabel("Internal Marks")
plt.show()

# Box Plot
plt.boxplot(df["Internal_Marks"])
plt.title("Box Plot of Internal Marks")
plt.ylabel("Marks")
plt.show()

print("\nObservations")
print("1. Most students with attendance above 75% passed.")
print("2. Students studying 4 to 6 hours scored higher internal marks.")
print("3. Students with low attendance mostly failed.")
print("4. Higher internal marks are strongly associated with passing.")
print("5. Attendance and study hours have a positive relationship with performance.")

print("\nConclusion")
print("Student performance is mainly influenced by attendance, study hours, and internal marks. Students with higher attendance and more study hours generally achieve better internal marks and pass the final examination.")
```

OUTPUT:

```
========== DATASET ==========
   Student_ID  Attendance  Study_Hours  Internal_Marks Result
0         S01          92            5              45   Pass
1         S02          55            2              24   Fail
2         S03          78            4              38   Pass
3         S04          60            3              28   Fail
4         S05          88            5              42   Pass
5         S06          47            1              20   Fail
6         S07          95            6              48   Pass
7         S08          72            3              35   Pass
8         S09          50            2              22   Fail
9         S10          83            4              40   Pass
10        S11          66            3              30   Fail
11        S12          90            5              44   Pass
12        S13          58            2              25   Fail
13        S14          76            4              36   Pass
14        S15          62            3              29   Fail
15        S16          85            5              41   Pass
16        S17          45            1              18   Fail
17        S18          80            4              39   Pass
18        S19          68            3              32   Pass
19        S20          53            2              23   Fail
```

1. Shape of Dataset
(20, 5)

2. Data Types
Student_ID object
Attendance int64
Study_Hours int64
Internal_Marks int64
Result object
dtype: object

3. Missing Values
Student_ID 0
Attendance 0
Study_Hours 0
Internal_Marks 0
Result 0
dtype: int64

Duplicate Values
0

Mean
Attendance 70.15
Study_Hours 3.35
Internal_Marks 32.95
dtype: float64

Median
Attendance 70.0
Study_Hours 3.0
Internal_Marks 33.5
dtype: float64

Minimum
Attendance 45
Study_Hours 1
Internal_Marks 18
dtype: int64

Maximum
Attendance 95
Study_Hours 6
Internal_Marks 48
dtype: int64

Standard Deviation
Attendance 15.938286
Study_Hours 1.424411
Internal_Marks 9.087151

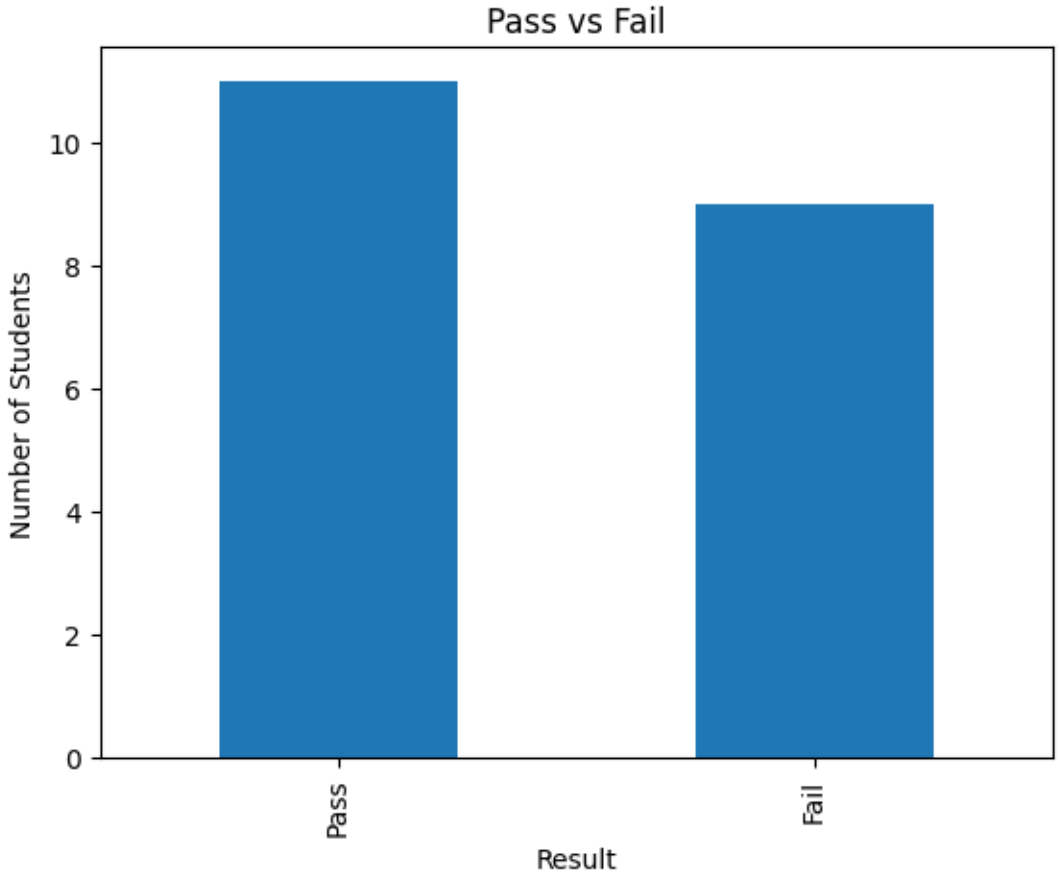
dtype: float64

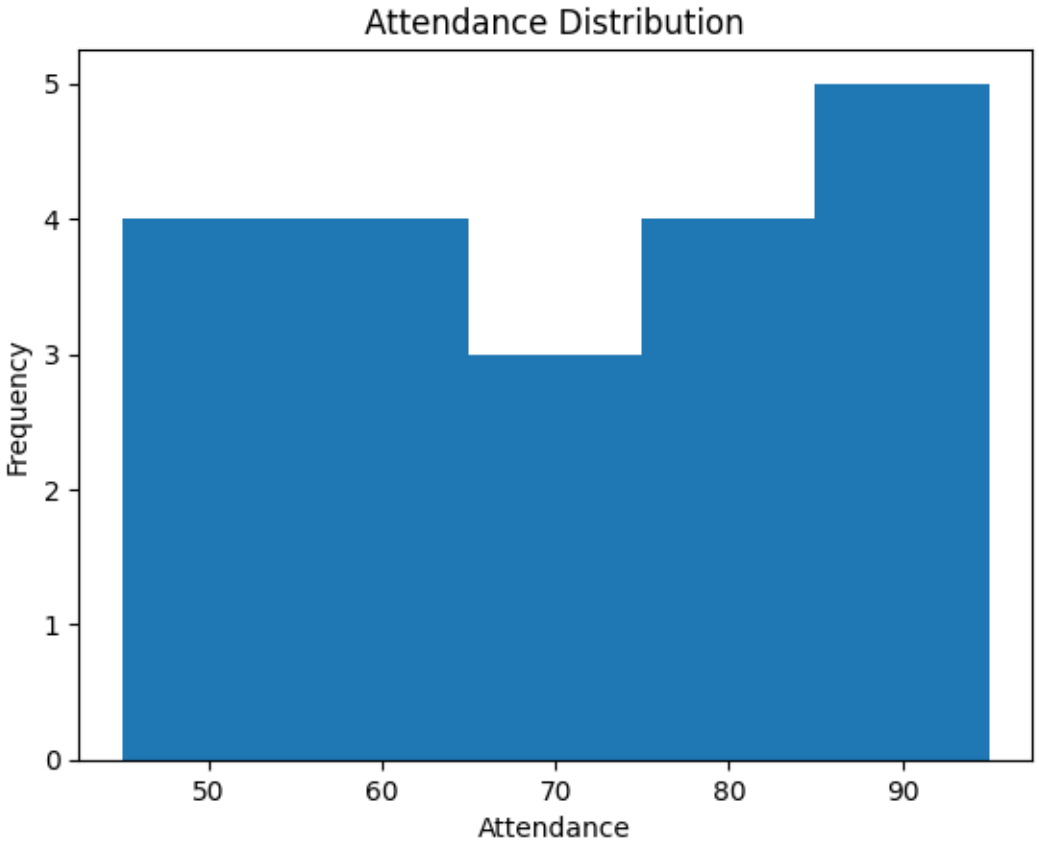
Correlation Matrix

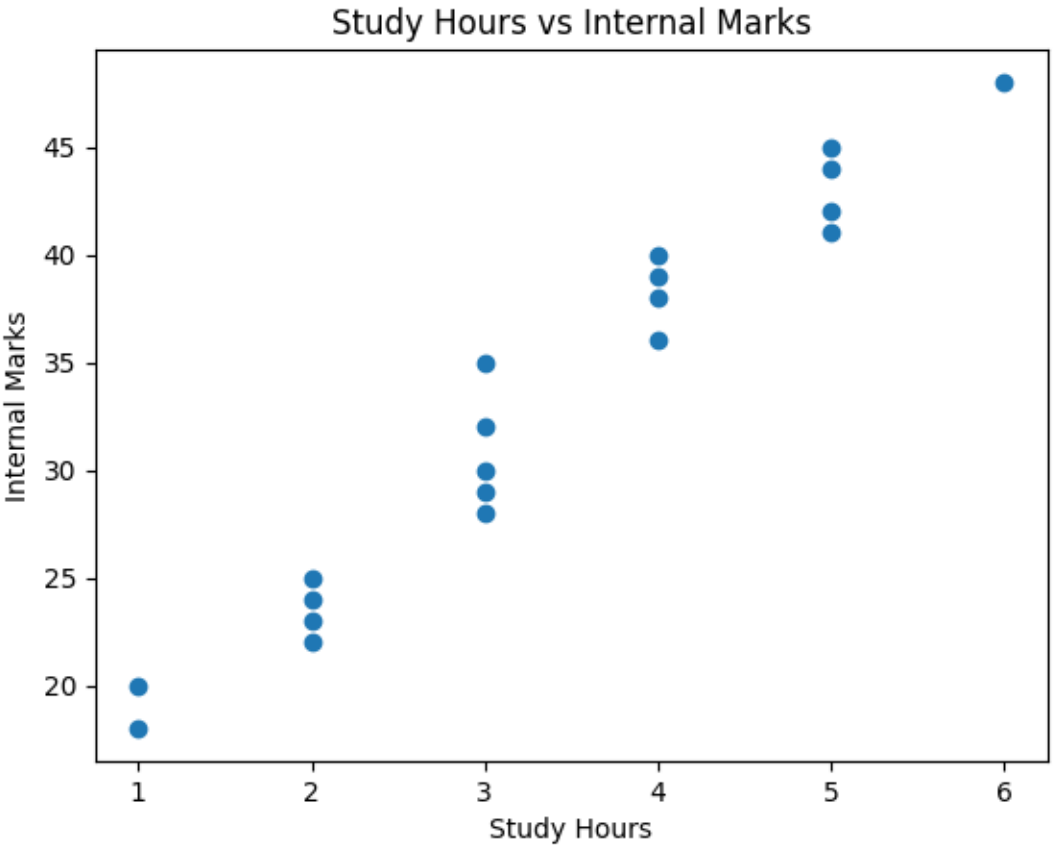
	Attendance	Study_Hours	Internal_Marks
Attendance	1.000000	0.975888	0.997569
Study_Hours	0.975888	1.000000	0.977299
Internal_Marks	0.997569	0.977299	1.000000

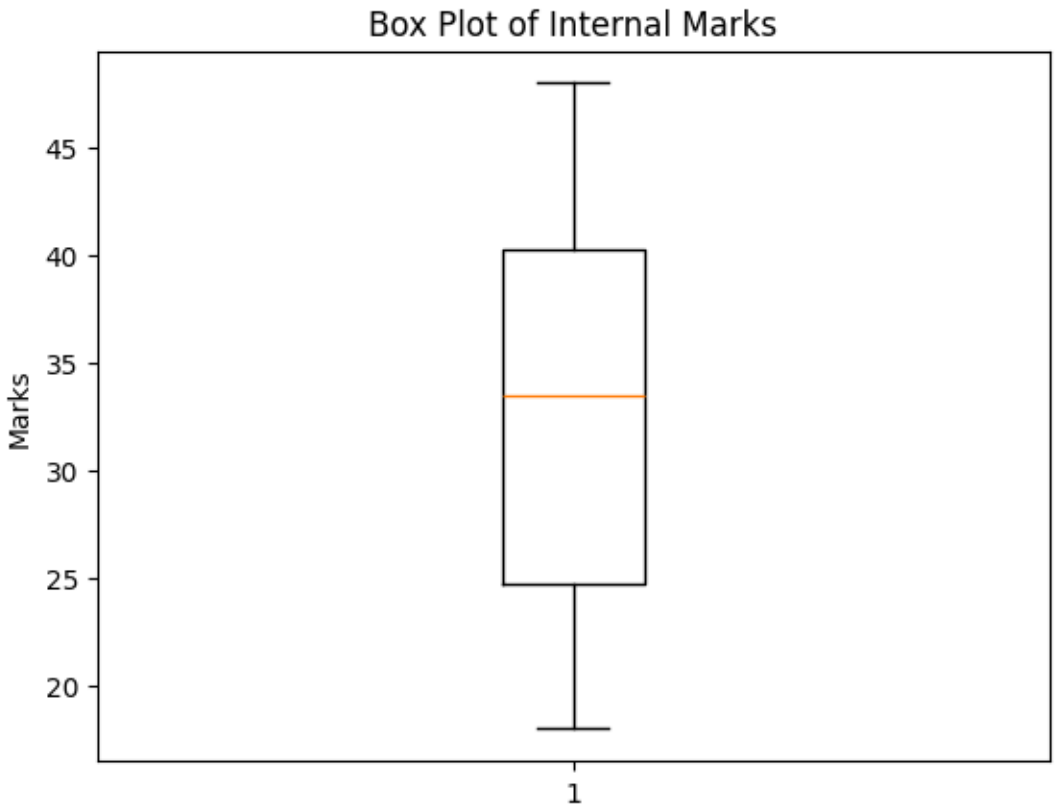
Average Values based on Result

Result	Attendance	Study_Hours	Internal_Marks
Fail	55.111111	2.111111	24.333333
Pass	82.454545	4.363636	40.000000




Attendance Distribution
Frequency
Attendance
0
1
2
3
4
5
50
60
70
80
90


Study Hours vs Internal Marks
Internal Marks
Study Hours
20
25
30
35
40
45
1
2
3
4
5
6



Observations
1. Most students with attendance above 75% passed.
2. Students studying 4 to 6 hours scored higher internal marks.
3. Students with low attendance mostly failed.
4. Higher internal marks are strongly associated with passing.
5. Attendance and study hours have a positive relationship with performance.

Conclusion
Student performance is mainly influenced by attendance, study hours, and internal marks. Students with higher attendance and more study hours generally achieve better internal marks and pass the final examination.

2. Healthcare Analytics Dataset
Question 2
A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No

P11	62	210	155	35	Yes
P12	38	115	122	25	No
P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes
P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Read CSV File
df = pd.read_csv("healthcare_data.csv")

# Display Dataset
print("========== DATASET ==========")
print(df)

# 1. Number of Rows and Columns
print("\n1. Shape of Dataset")
print(df.shape)

# 2. Data Types
print("\n2. Data Types")
print(df.dtypes)

# 3. Missing Values
print("\n3. Missing Values")
print(df.isnull().sum())

# Duplicate Values
print("\nDuplicate Values")
print(df.duplicated().sum())

# 4. Statistical Measures

print("\nMean")
print(df.mean(numeric_only=True))

print("\nMedian")
print(df.median(numeric_only=True))

print("\nMinimum")
print(df.min(numeric_only=True))

print("\nMaximum")
```

```
print(df.max(numeric_only=True))

print("\nStandard Deviation")
print(df.std(numeric_only=True))

# Correlation Matrix
print("\nCorrelation Matrix")
print(df[["Age","Sugar_Level","BP","BMI"]].corr())

# Average Values based on Disease Status
print("\nAverage Values based on Disease Status")
print(df.groupby("Disease_Status")[["Age","Sugar_Level","BP","BMI"]].mean())


# Bar Chart
df["Disease_Status"].value_counts().plot(kind="bar")
plt.title("Disease Status")
plt.xlabel("Disease Status")
plt.ylabel("Number of Patients")
plt.show()

# Histogram
plt.hist(df["Sugar_Level"], bins=5)
plt.title("Sugar Level Distribution")
plt.xlabel("Sugar Level")
plt.ylabel("Frequency")
plt.show()

# Scatter Plot
plt.scatter(df["Age"], df["Sugar_Level"])
plt.title("Age vs Sugar Level")
plt.xlabel("Age")
plt.ylabel("Sugar Level")
plt.show()

# Box Plot
plt.boxplot(df["BMI"])
plt.title("Box Plot of BMI")
plt.ylabel("BMI")
plt.show()

print("\nObservations")
print("1. Patients with higher sugar levels are more likely to have the disease.")
print("2. Higher BMI is commonly observed among patients with disease.")
print("3. Older patients generally have higher blood pressure and sugar levels.")
print("4. Patients without disease have lower BMI and sugar levels.")
print("5. Age, Sugar Level, BP, and BMI influence disease status.")


print("\nConclusion")
```

print("Patients with high blood sugar, high BMI, and high blood pressure have a greater risk of disease. Maintaining healthy lifestyle factors may reduce disease risk.")

OUTPUT:

```
========== DATASET ==========
  Patient_ID  Age  Sugar_Level   BP  BMI Disease_Status
0        P01   45          145  130   28            Yes
1        P02   32           98  118   23             No
2        P03   55          180  145   31            Yes
3        P04   28           90  110   22             No
4        P05   60          200  150   34            Yes
5        P06   40          120  120   26             No
```

1. Shape of Dataset
(6, 6)

2. Data Types
Patient_ID object
Age int64
Sugar_Level int64
BP int64
BMI int64
Disease_Status object
dtype: object

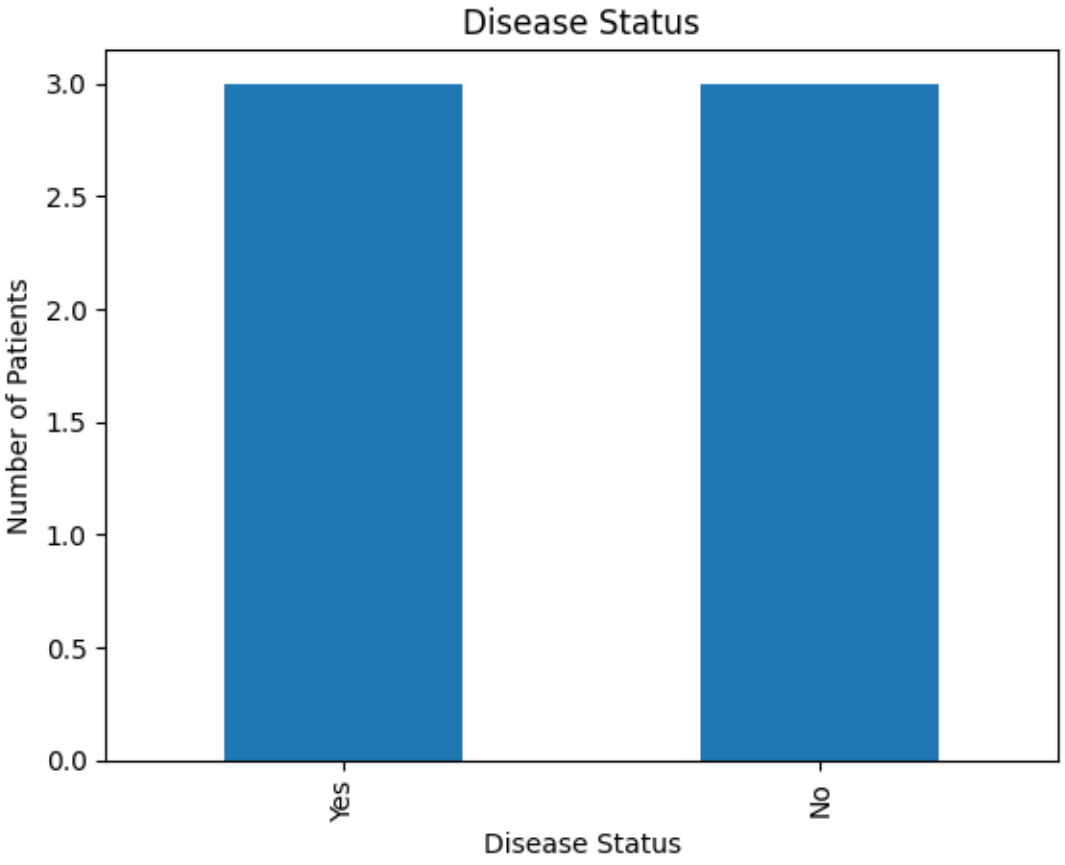
3. Missing Values
Patient_ID 0
Age 0
Sugar_Level 0
BP 0
BMI 0
Disease_Status 0
dtype: int64

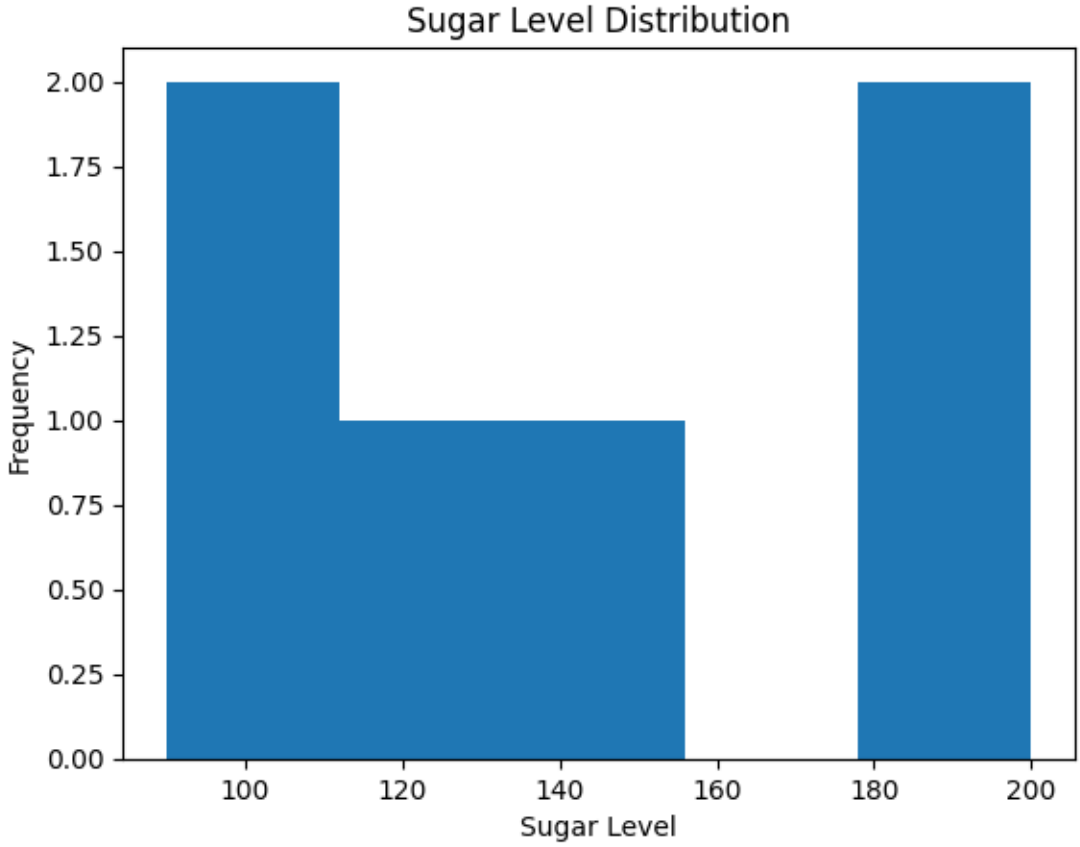
Duplicate Values
0

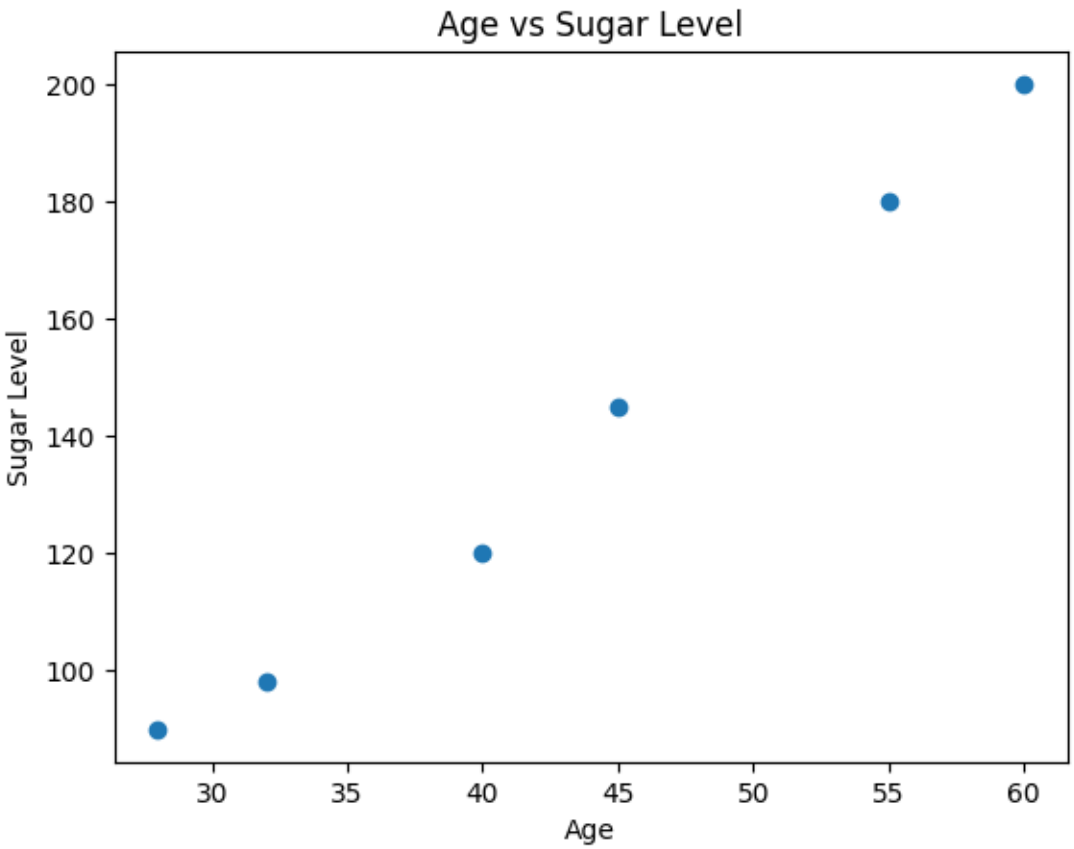
Mean
Age 43.333333
Sugar_Level 138.833333
BP 128.833333
BMI 27.333333
dtype: float64

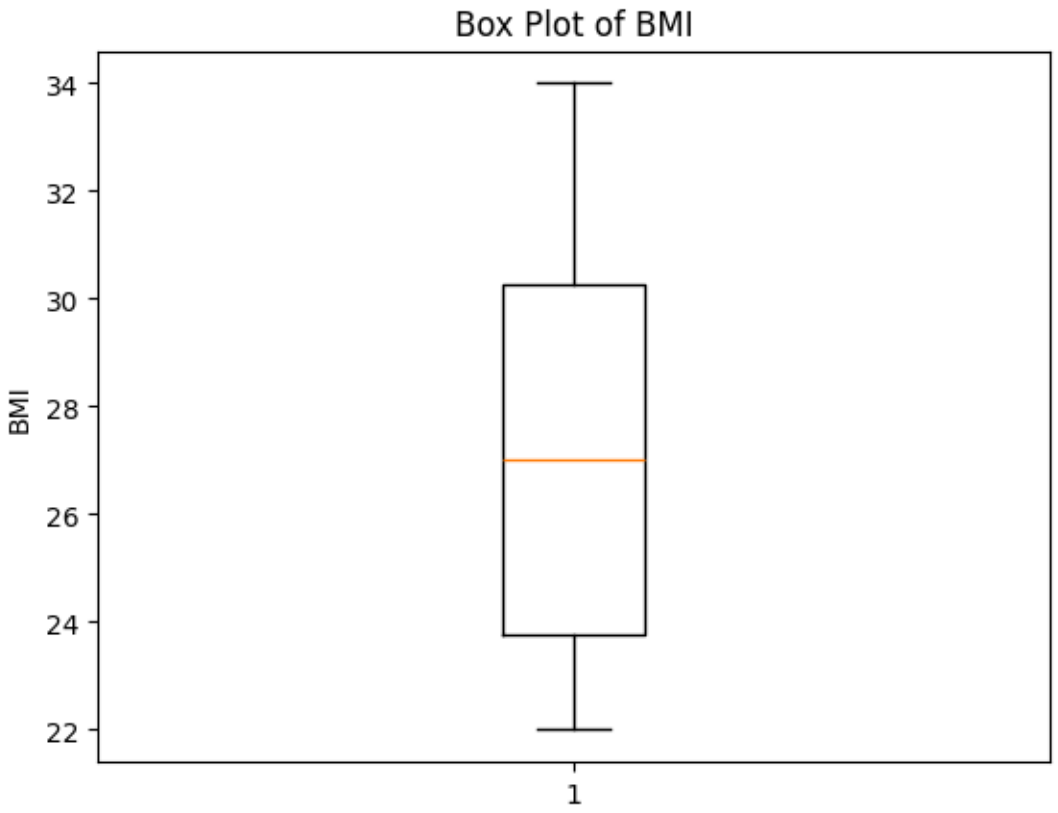
Median
Age 42.5
Sugar_Level 132.5
BP 125.0

```
BMI            27.0
dtype: float64

Minimum
Age            28
Sugar_Level    90
BP            110
BMI            22
dtype: int64

Maximum
Age            60
Sugar_Level   200
BP            150
BMI            34
dtype: int64

Standard Deviation
Age           12.580408
Sugar_Level   44.454096
BP            15.879757
BMI            4.633213
dtype: float64

Correlation Matrix
                Age  Sugar_Level        BP       BMI
Age          1.000000     0.995737  0.985448  0.996208
Sugar_Level  0.995737     1.000000  0.990433  0.995637
BP           0.985448     0.990433  1.000000  0.979509
BMI          0.996208     0.995637  0.979509  1.000000

Average Values based on Disease Status
                    Age  Sugar_Level         BP        BMI
Disease_Status
No            33.333333   102.666667  116.000000  23.666667
Yes           53.333333   175.000000  141.666667  31.000000
```


Disease Status
Number of Patients
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Yes
No
Disease Status


Sugar Level Distribution
Frequency
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
100
120
140
160
180
200
Sugar Level





Observations

1. Patients with higher sugar levels are more likely to have the disease.
2. Higher BMI is commonly observed among patients with disease.
3. Older patients generally have higher blood pressure and sugar levels.

4. Patients without disease have lower BMI and sugar levels.
5. Age, Sugar Level, BP, and BMI influence disease status.

Conclusion
Patients with high blood sugar, high BMI, and high blood pressure have a greater risk of disease. Maintaining healthy lifestyle factors may reduce disease risk.

3. Retail Sales Dataset
Question 3
A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

CODE:
```
import pandas as pd
import matplotlib.pyplot as plt

# Read CSV File
df = pd.read_csv("retail_sales.csv")

# Display Dataset
print("========== DATASET ==========")
print(df)

# 1. Number of Rows and Columns
print("\n1. Shape of Dataset")
print(df.shape)

# 2. Data Types
```

```
print("\n2. Data Types")
print(df.dtypes)

# 3. Missing Values
print("\n3. Missing Values")
print(df.isnull().sum())

# Duplicate Values
print("\nDuplicate Values")
print(df.duplicated().sum())

# 4. Statistical Measures

print("\nMean")
print(df.mean(numeric_only=True))

print("\nMedian")
print(df.median(numeric_only=True))

print("\nMinimum")
print(df.min(numeric_only=True))

print("\nMaximum")
print(df.max(numeric_only=True))

print("\nStandard Deviation")
print(df.std(numeric_only=True))

# Correlation Matrix
print("\nCorrelation Matrix")
print(df[["Price","Quantity_Sold","Discount","Revenue"]].corr())

# Average Values based on Category
print("\nAverage Values based on Category")
print(df.groupby("Category")[["Price","Quantity_Sold","Discount","Revenue"]].mean())

# Bar Chart
df["Category"].value_counts().plot(kind="bar")
plt.title("Products by Category")
plt.xlabel("Category")
plt.ylabel("Number of Products")
plt.show()

# Histogram
plt.hist(df["Revenue"], bins=5)
plt.title("Revenue Distribution")
plt.xlabel("Revenue")
plt.ylabel("Frequency")
plt.show()
```

```
# Scatter Plot
plt.scatter(df["Quantity_Sold"], df["Revenue"])
plt.title("Quantity Sold vs Revenue")
plt.xlabel("Quantity Sold")
plt.ylabel("Revenue")
plt.show()

# Box Plot
plt.boxplot(df["Revenue"])
plt.title("Box Plot of Revenue")
plt.ylabel("Revenue")
plt.show()

print("\nObservations")
print("1. Fruits generate the highest average revenue.")
print("2. Products with higher quantity sold generally earn higher revenue.")
print("3. Grocery products have the highest average selling price.")
print("4. Dairy products receive lower discounts compared to other categories.")
print("5. Revenue is mainly influenced by both price and quantity sold.")

print("\nConclusion")
print("The supermarket's revenue is mainly affected by product price and quantity sold. Fruits contribute the highest revenue, while increasing sales quantity and maintaining suitable pricing can improve overall business performance.")
```

OUTPUT:

```
========== DATASET ==========
   Product_ID    Category  Price  Quantity_Sold  Discount  Revenue
0        PR01     Grocery     50             20         5     1000
1        PR02      Snacks     30             35         3     1050
2        PR03       Dairy     45             18         2      810
3        PR04      Fruits     60             25         5     1500
4        PR05  Vegetables     40             30         4     1200
5        PR06     Grocery     80             15         6     1200
6        PR07      Snacks     25             40         3     1000
7        PR08       Dairy     55             16         2      880
8        PR09      Fruits     70             22         5     1540
9        PR10  Vegetables     35             28         4      980
10       PR11     Grocery     90             12         6     1080
11       PR12      Snacks     20             45         2      900
12       PR13       Dairy     65             14         3      910
13       PR14      Fruits     75             20         5     1500
14       PR15  Vegetables     45             26         4     1170
15       PR16     Grocery    100             10         7     1000
16       PR17      Snacks     35             32         3     1120
17       PR18       Dairy     50             19         2      950
18       PR19      Fruits     80             18         6     1440
19       PR20  Vegetables     30             35         3     1050
```

1. Shape of Dataset
(20, 6)

2. Data Types
Product_ID object
Category object

Price int64
Quantity_Sold int64
Discount int64
Revenue int64
dtype: object

3. Missing Values
Product_ID 0
Category 0
Price 0
Quantity_Sold 0
Discount 0
Revenue 0
dtype: int64

Duplicate Values
0

Mean
Price 54.0
Quantity_Sold 24.0
Discount 4.0
Revenue 1114.0
dtype: float64

Median
Price 50.0
Quantity_Sold 21.0
Discount 4.0
Revenue 1050.0
dtype: float64

Minimum
Price 20
Quantity_Sold 10
Discount 2
Revenue 810
dtype: int64

Maximum
Price 100
Quantity_Sold 45
Discount 7
Revenue 1540
dtype: int64

Standard Deviation
Price 22.803509
Quantity_Sold 9.673621
Discount 1.555973

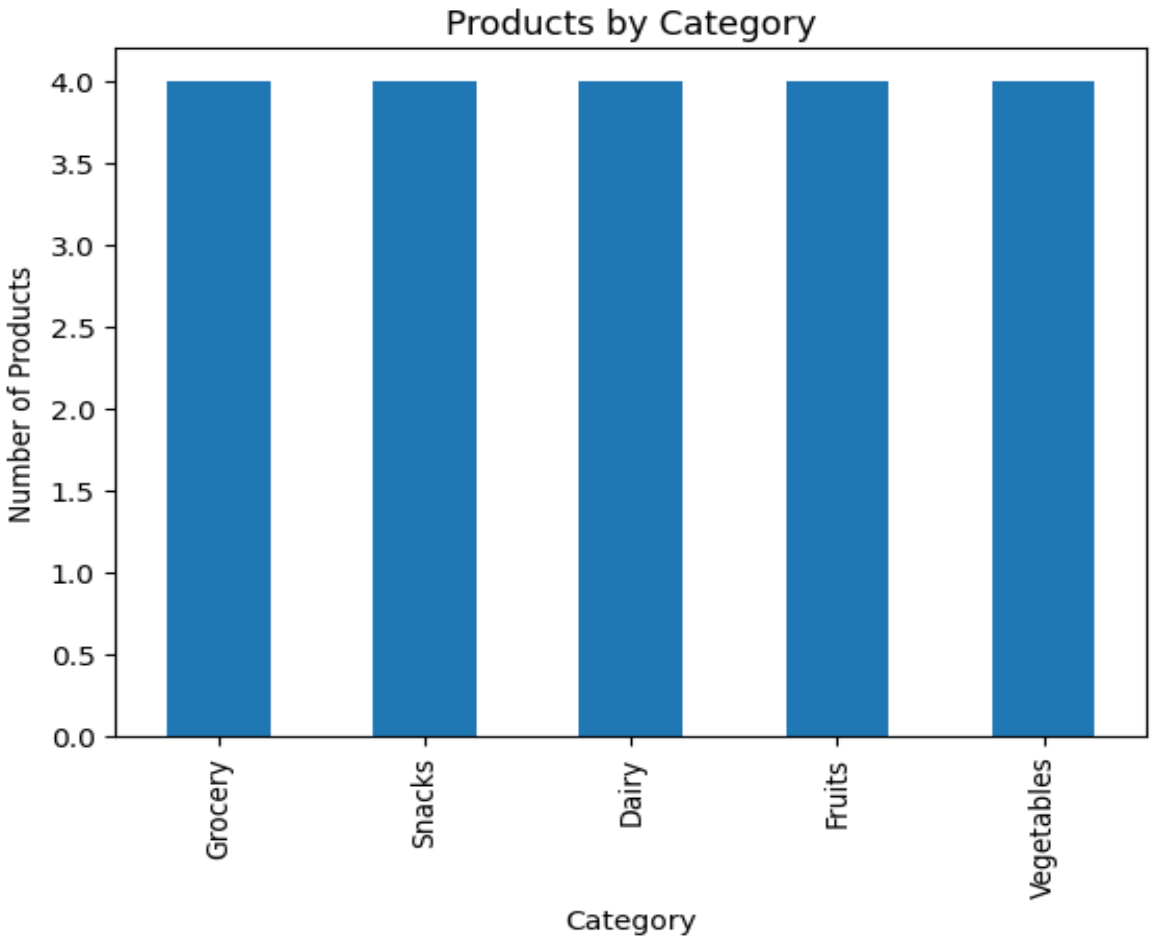
Revenue 221.416493
dtype: float64

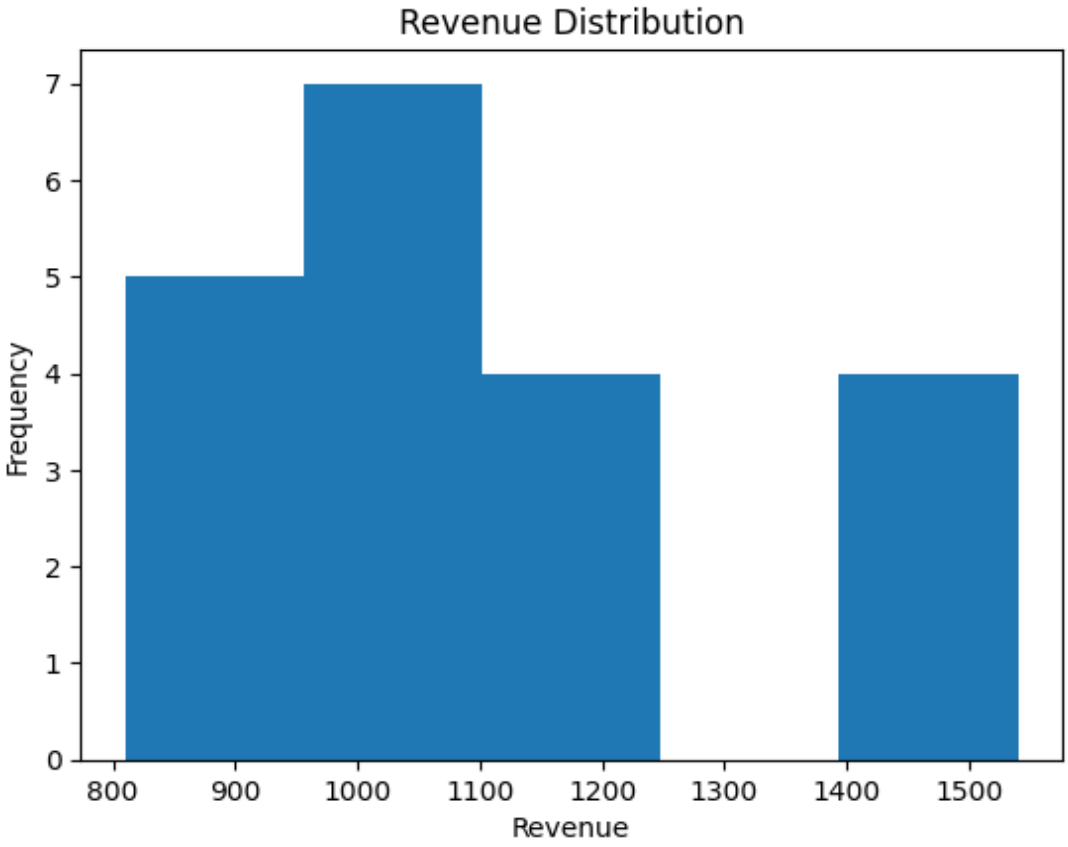
Correlation Matrix

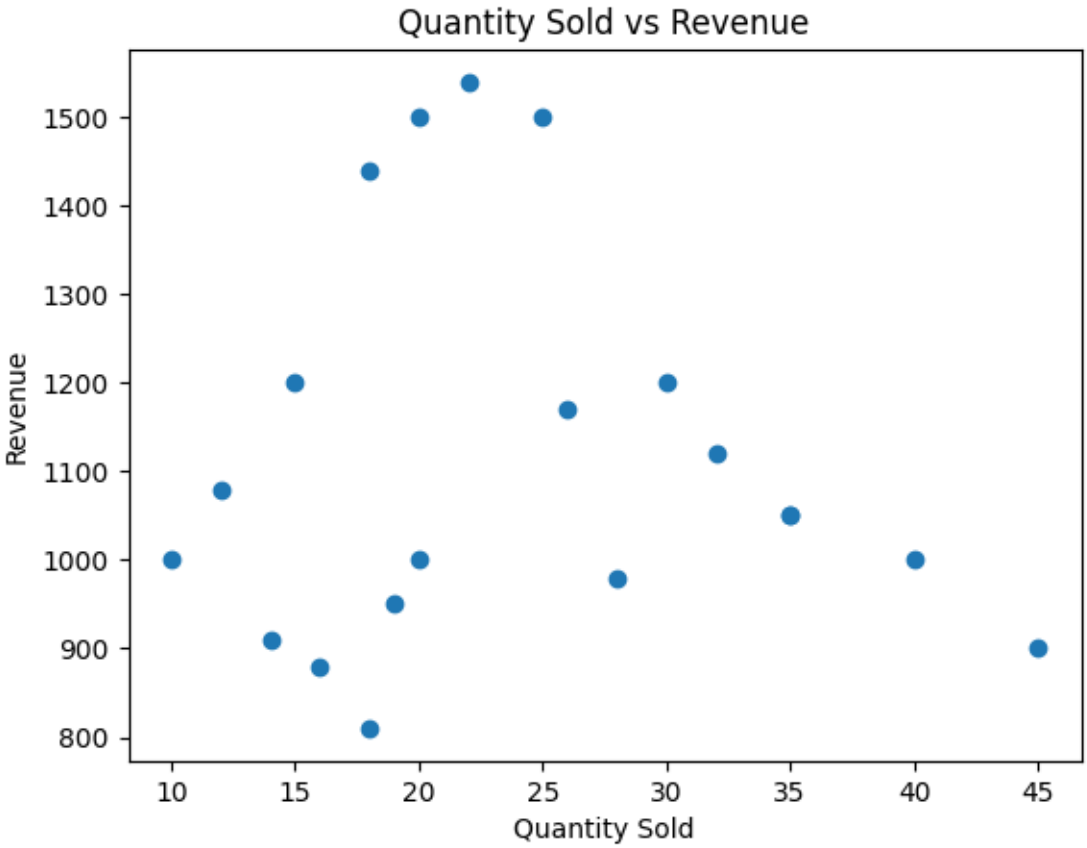
	Price	Quantity_Sold	Discount	Revenue
Price	1.000000	-0.868474	0.786174	0.388086
Quantity_Sold	-0.868474	1.000000	-0.507018	-0.081335
Discount	0.786174	-0.507018	1.000000	0.575938
Revenue	0.388086	-0.081335	0.575938	1.000000

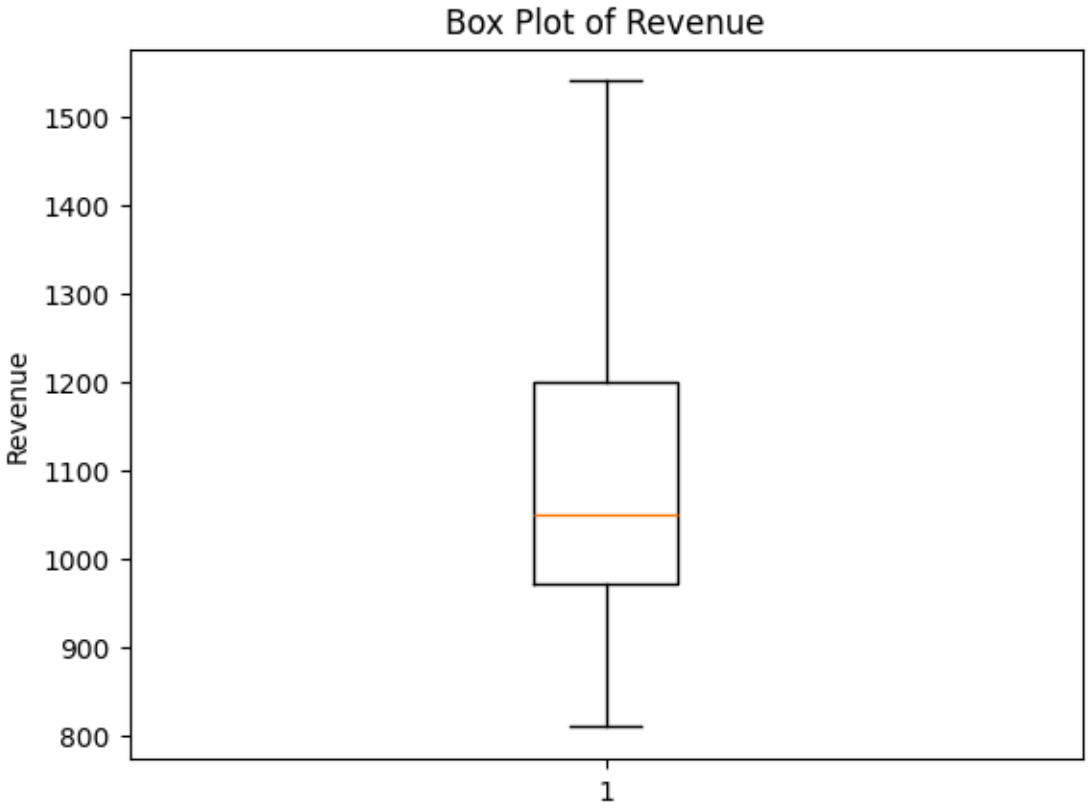
Average Values based on Category

Category	Price	Quantity_Sold	Discount	Revenue
Dairy	53.75	16.75	2.25	887.5
Fruits	71.25	21.25	5.25	1495.0
Grocery	80.00	14.25	6.00	1070.0
Snacks	27.50	38.00	2.75	1017.5
Vegetables	37.50	29.75	3.75	1100.0




Revenue Distribution
Frequency
Revenue
0
1
2
3
4
5
6
7
800
900
1000
1100
1200
1300
1400
1500


Quantity Sold vs Revenue
Revenue
Quantity Sold
800
900
1000
1100
1200
1300
1400
1500
10
15
20
25
30
35
40
45



Observations
1. Fruits generate the highest average revenue.
2. Products with higher quantity sold generally earn higher revenue.
3. Grocery products have the highest average selling price.
4. Dairy products receive lower discounts compared to other categories.
5. Revenue is mainly influenced by both price and quantity sold.

Conclusion
The supermarket's revenue is mainly affected by product price and quantity sold. Fruits contribute the highest revenue, while increasing sales quantity and maintaining suitable pricing can improve overall business performance.

4. Banking Loan Approval Dataset
A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected
C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved
C10	24000	570	130000	Unemployed	Rejected

C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected
C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Read CSV File
df = pd.read_csv("banking_loan.csv")

# Display Dataset
print("========== DATASET ==========")
print(df)

# 1. Number of Rows and Columns
print("\n1. Shape of Dataset")
print(df.shape)

# 2. Data Types
print("\n2. Data Types")
print(df.dtypes)

# 3. Missing Values
print("\n3. Missing Values")
print(df.isnull().sum())

# Duplicate Values
print("\nDuplicate Values")
print(df.duplicated().sum())

# 4. Statistical Measures

print("\nMean")
print(df.mean(numeric_only=True))

print("\nMedian")
print(df.median(numeric_only=True))

print("\nMinimum")
print(df.min(numeric_only=True))
```

```
print("\nMaximum")
print(df.max(numeric_only=True))

print("\nStandard Deviation")
print(df.std(numeric_only=True))

# Correlation Matrix
print("\nCorrelation Matrix")
print(df[["Income","Credit_Score","Loan_Amount"]].corr())

# Average Values based on Loan Status
print("\nAverage Values based on Loan Status")
print(df.groupby("Loan_Status")[["Income","Credit_Score","Loan_Amount"]].mean())

# ---------- CHARTS ----------

# Bar Chart
df["Loan_Status"].value_counts().plot(kind="bar")
plt.title("Loan Approval Status")
plt.xlabel("Loan Status")
plt.ylabel("Number of Customers")
plt.show()

# Histogram
plt.hist(df["Income"], bins=5)
plt.title("Income Distribution")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()

# Scatter Plot
plt.scatter(df["Credit_Score"], df["Loan_Amount"])
plt.title("Credit Score vs Loan Amount")
plt.xlabel("Credit Score")
plt.ylabel("Loan Amount")
plt.show()

# Box Plot
plt.boxplot(df["Income"])
plt.title("Box Plot of Income")
plt.ylabel("Income")
plt.show()

# ---------- OBSERVATIONS ----------

print("\nObservations")
print("1. Customers with higher income are mostly approved for loans.")
print("2. Higher credit scores are associated with loan approval.")
print("3. Salaried customers have a higher approval rate than other employment types.")
print("4. Customers with low income and low credit scores are mostly rejected.")
```

```
print("5. Income and credit score positively influence the loan amount approved.")

# ---------- CONCLUSION ----------

print("\nConclusion")
print("Loan approval mainly depends on income and credit score. Customers with stable salaried employment, higher income, and good credit scores are more likely to receive loan approval.")
```

OUTPUT:

```
========== DATASET ==========
   Customer_ID  Income  Credit_Score  Loan_Amount Employment_Type Loan_Status
0          C01   45000           720       200000        Salaried    Approved
1          C02   25000           580       150000   Self-employed    Rejected
2          C03   60000           750       300000        Salaried    Approved
3          C04   22000           560       120000      Unemployed    Rejected
4          C05   50000           710       250000        Salaried    Approved
5          C06   30000           600       180000   Self-employed    Rejected
6          C07   70000           780       350000        Salaried    Approved
7          C08   28000           590       160000   Self-employed    Rejected
8          C09   55000           730       270000        Salaried    Approved
9          C10   24000           570       130000      Unemployed    Rejected
10         C11   65000           760       320000        Salaried    Approved
11         C12   35000           620       200000   Self-employed    Rejected
12         C13   48000           700       240000        Salaried    Approved
13         C14   26000           585       140000      Unemployed    Rejected
14         C15   75000           790       400000        Salaried    Approved
15         C16   32000           610       190000   Self-employed    Rejected
16         C17   52000           725       260000        Salaried    Approved
17         C18   27000           575       150000   Self-employed    Rejected
18         C19   68000           770       330000        Salaried    Approved
19         C20   23000           555       125000      Unemployed    Rejected
```

1. Shape of Dataset
(20, 6)

2. Data Types
Customer_ID object
Income int64
Credit_Score int64
Loan_Amount int64
Employment_Type object
Loan_Status object
dtype: object

3. Missing Values
Customer_ID 0
Income 0
Credit_Score 0
Loan_Amount 0
Employment_Type 0
Loan_Status 0
dtype: int64

Duplicate Values
0

Mean
Income 43000.0
Credit_Score 664.0
Loan_Amount 223250.0
dtype: float64

Median
Income 40000.0
Credit_Score 660.0
Loan_Amount 200000.0
dtype: float64

Minimum
Income 22000
Credit_Score 555
Loan_Amount 120000
dtype: int64

Maximum
Income 75000
Credit_Score 790
Loan_Amount 400000
dtype: int64

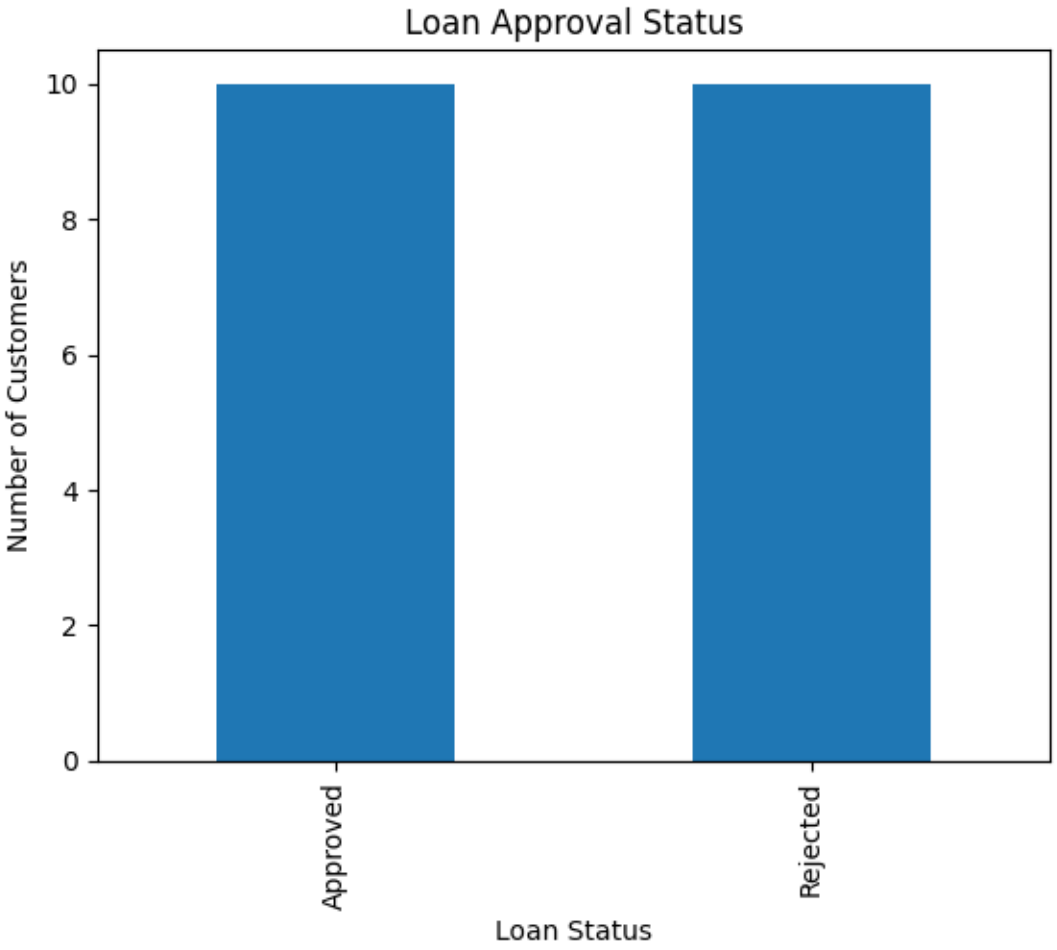
Standard Deviation
Income 17923.815383
Credit_Score 85.526235
Loan_Amount 83733.207272
dtype: float64

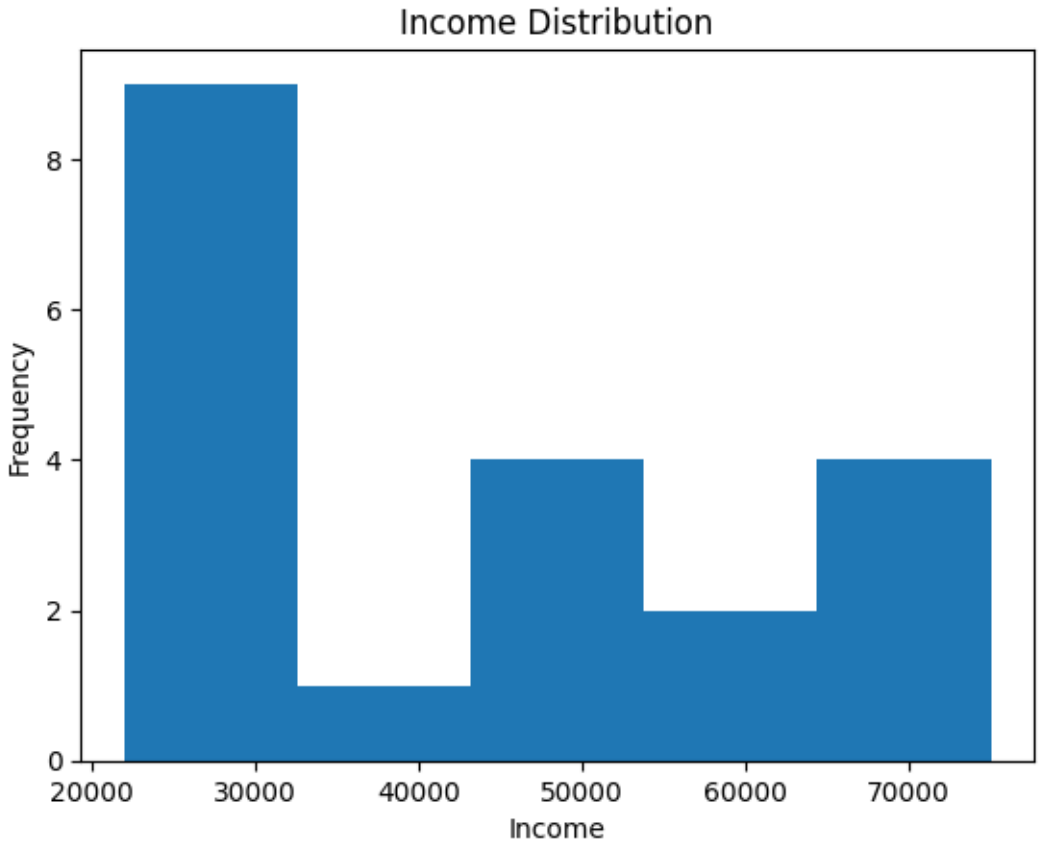
Correlation Matrix

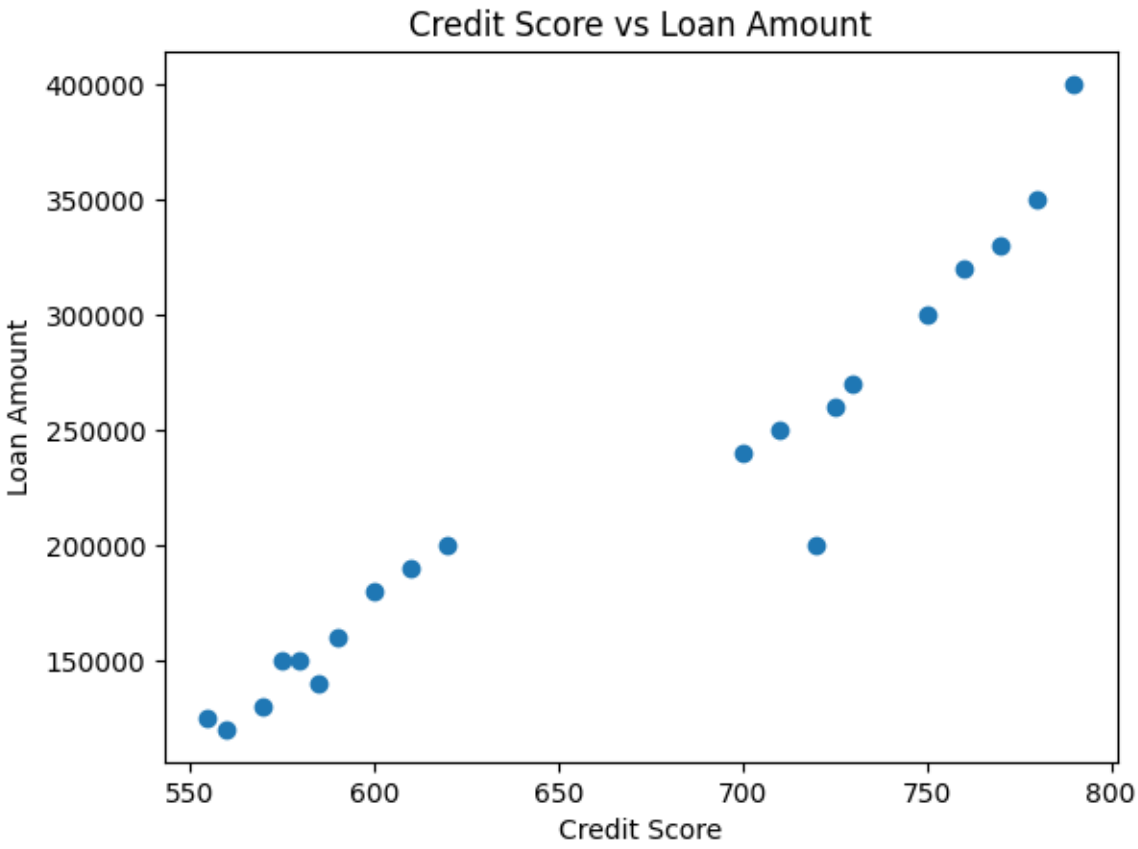
	Income	Credit_Score	Loan_Amount
Income	1.000000	0.983995	0.987882
Credit_Score	0.983995	1.000000	0.953689
Loan_Amount	0.987882	0.953689	1.000000

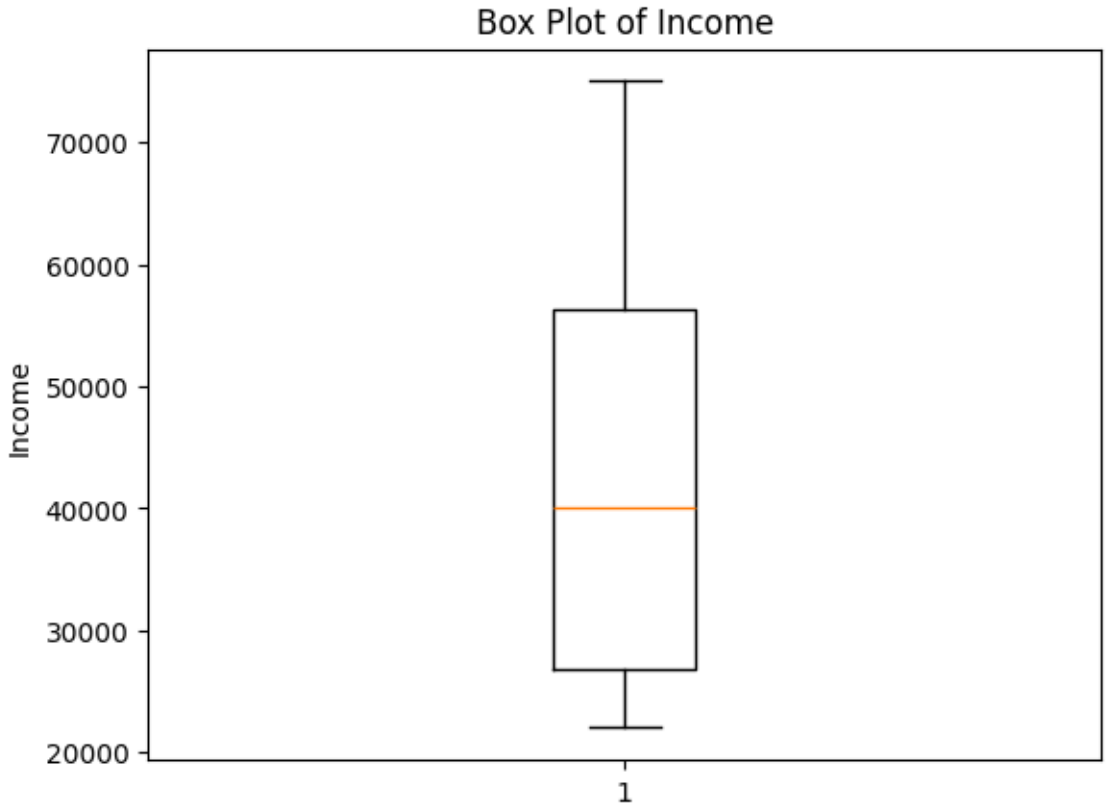
Average Values based on Loan Status

Loan_Status	Income	Credit_Score	Loan_Amount
Approved	58800.0	743.5	292000.0
Rejected	27200.0	584.5	154500.0


Loan Approval Status
Number of Customers
0
2
4
6
8
10
Approved
Rejected
Loan Status


Income Distribution
Frequency
0
2
4
6
8
20000
30000
40000
50000
60000
70000
Income





Observations

1. Customers with higher income are mostly approved for loans.
2. Higher credit scores are associated with loan approval.
3. Salaried customers have a higher approval rate than other employment types.

4. Customers with low income and low credit scores are mostly rejected.
5. Income and credit score positively influence the loan amount approved.

Conclusion
Loan approval mainly depends on income and credit score. Customers with stable salaried employment, higher income, and good credit scores are more likely to receive loan approval.

5. Agriculture Crop Yield Dataset
Question 5
An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800
F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

CODE:
```
import pandas as pd
import matplotlib.pyplot as plt
# Read CSV
df = pd.read_csv("crop_yield.csv")
# 1. Number of Rows and Columns
print("Shape of Dataset:", df.shape)
# Display Dataset
print("\nDataset:")
print(df)
# 2. Data Types
print("\nData Types:")
print(df.dtypes)
# 3. Missing Values
print("\nMissing Values:")
```

```
print(df.isnull().sum())
# Duplicate Values
print("\nDuplicate Values:", df.duplicated().sum())
# 4. Statistical Measures
print("\nMean:")
print(df.mean(numeric_only=True))
print("\nMedian:")
print(df.median(numeric_only=True))
print("\nMinimum:")
print(df.min(numeric_only=True))
print("\nMaximum:")
print(df.max(numeric_only=True))
print("\nStandard Deviation:")
print(df.std(numeric_only=True))
# Correlation Matrix
print("\nCorrelation Matrix:")
print(df[["Rainfall_mm","Temperature","Fertilizer_kg","Crop_Yield_kg"]].corr())
# Average Crop Yield by Soil Type
print("\nAverage Crop Yield by Soil Type:")
print(df.groupby("Soil_Type")["Crop_Yield_kg"].mean())
# ---------- Charts ----------
# Bar Chart
df.groupby("Soil_Type")["Crop_Yield_kg"].mean().plot(kind="bar")
plt.title("Average Crop Yield by Soil Type")
plt.xlabel("Soil Type")
plt.ylabel("Crop Yield (kg)")
plt.show()
# Histogram
plt.hist(df["Crop_Yield_kg"], bins=5)
plt.title("Crop Yield Distribution")
plt.xlabel("Crop Yield (kg)")
plt.ylabel("Frequency")
plt.show()
# Scatter Plot
plt.scatter(df["Rainfall_mm"], df["Crop_Yield_kg"])
plt.title("Rainfall vs Crop Yield")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (kg)")
plt.show()
# Box Plot
plt.boxplot(df["Crop_Yield_kg"])
plt.title("Box Plot of Crop Yield")
plt.ylabel("Crop Yield (kg)")
plt.show()
# ---------- Observations ----------
print("\nObservations:")
print("1. Loamy soil has the highest average crop yield.")
print("2. Higher rainfall generally results in higher crop yield.")
print("3. Increased fertilizer usage is associated with higher crop production.")
print("4. Lower temperatures with adequate rainfall produce better yields.")
```

```
print("5. Sandy soil shows the lowest crop yield among all soil types.")
# ---------- Conclusion ----------
print("\nConclusion:")
print("Rainfall, fertilizer usage, and soil type significantly influence crop yield. Loamy soil combined with adequate rainfall and fertilizer produces the highest agricultural output.")
```

OUTPUT:

Dataset:

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
0	F01	850	28	60	Loamy	3200
1	F02	600	32	45	Sandy	2100
2	F03	900	27	65	Loamy	3500
3	F04	500	34	40	Sandy	1800
4	F05	780	29	55	Clay	3000
5	F06	650	31	48	Sandy	2300
6	F07	920	26	70	Loamy	3700
7	F08	720	30	52	Clay	2800
8	F09	550	33	42	Sandy	1900
9	F10	800	28	58	Loamy	3100
10	F11	880	27	62	Loamy	3400
11	F12	620	32	46	Sandy	2200
12	F13	760	29	54	Clay	2950
13	F14	940	26	72	Loamy	3800
14	F15	580	33	44	Sandy	2000
15	F16	820	28	60	Clay	3150
16	F17	700	30	50	Clay	2700
17	F18	960	25	75	Loamy	3900
18	F19	540	34	41	Sandy	1850
19	F20	790	29	56	Clay	3050

Shape of Dataset: (20, 6)

Data Types:
Farm_ID object
Rainfall_mm int64
Temperature int64
Fertilizer_kg int64
Soil_Type object
Crop_Yield_kg int64
dtype: object

Missing Values:
Farm_ID 0
Rainfall_mm 0
Temperature 0
Fertilizer_kg 0
Soil_Type 0
Crop_Yield_kg 0
dtype: int64

Duplicate Values: 0
Mean:
Rainfall_mm 743.00
Temperature 29.55
Fertilizer_kg 54.75
Crop_Yield_kg 2820.00
dtype: float64

Median:
Rainfall_mm 770.0
Temperature 29.0
Fertilizer_kg 54.5
Crop_Yield_kg 2975.0
dtype: float64

Minimum:
Rainfall_mm 500
Temperature 25
Fertilizer_kg 40
Crop_Yield_kg 1800
dtype: int64

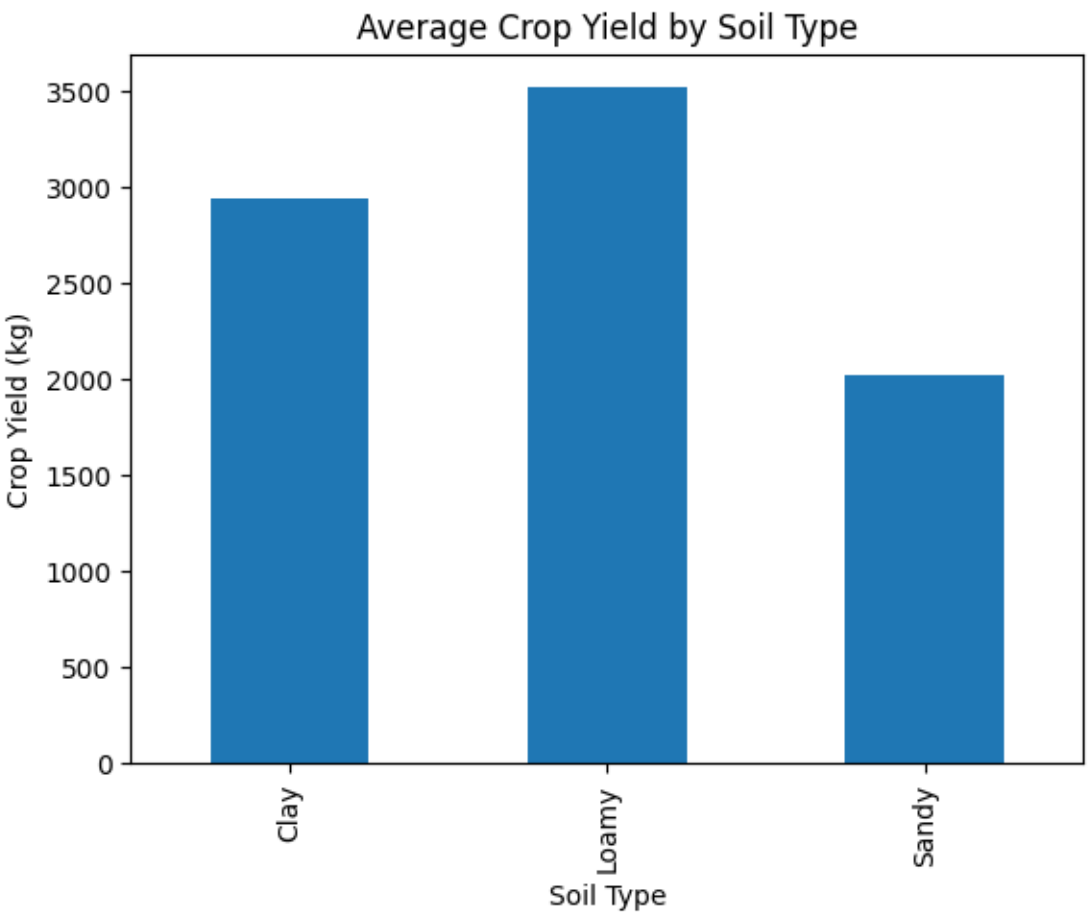
Maximum:
Rainfall_mm 960
Temperature 34
Fertilizer_kg 75
Crop_Yield_kg 3900
dtype: int64

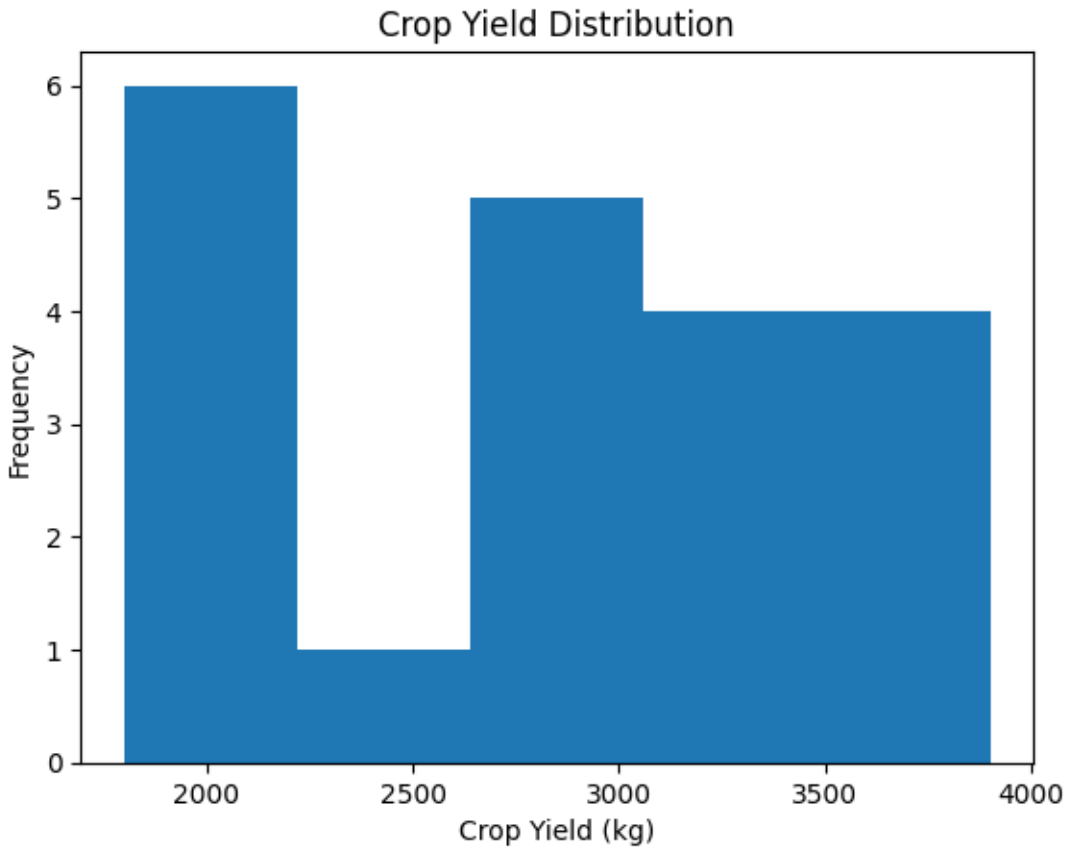
Standard Deviation:
Rainfall_mm 144.335136
Temperature 2.762055
Fertilizer_kg 10.507516
Crop_Yield_kg 681.407213
dtype: float64

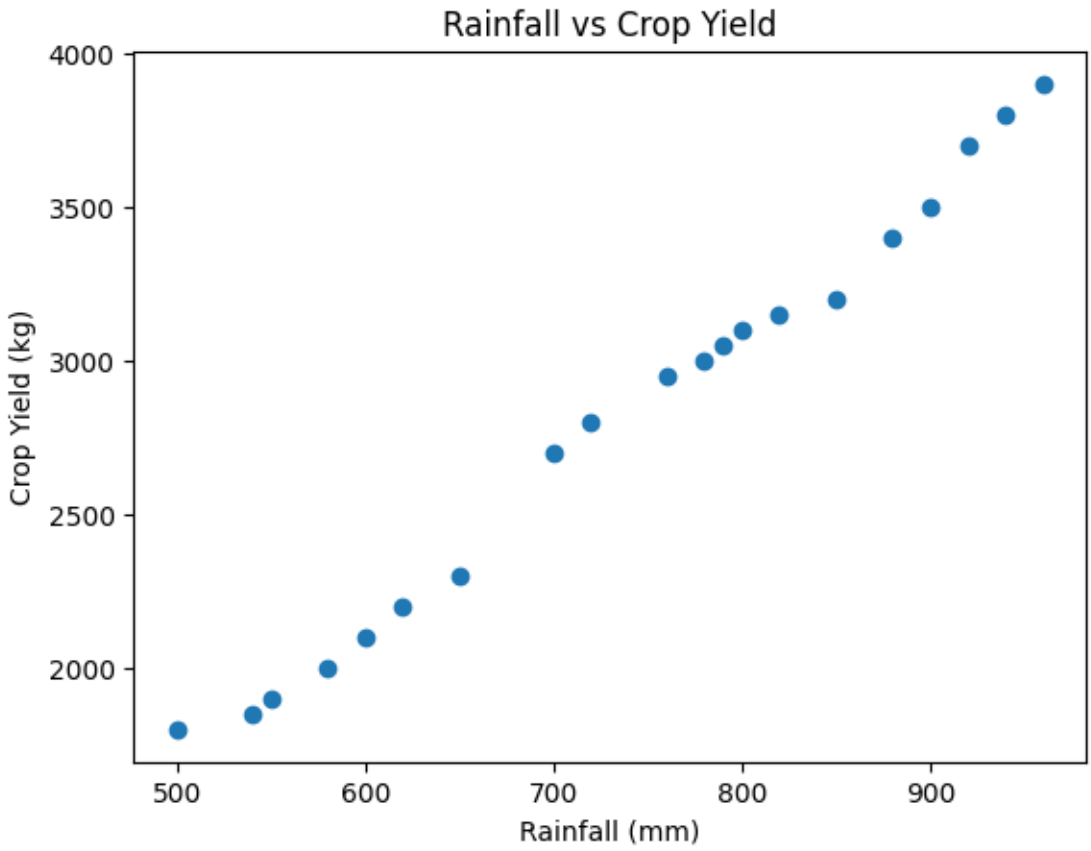
Correlation Matrix:

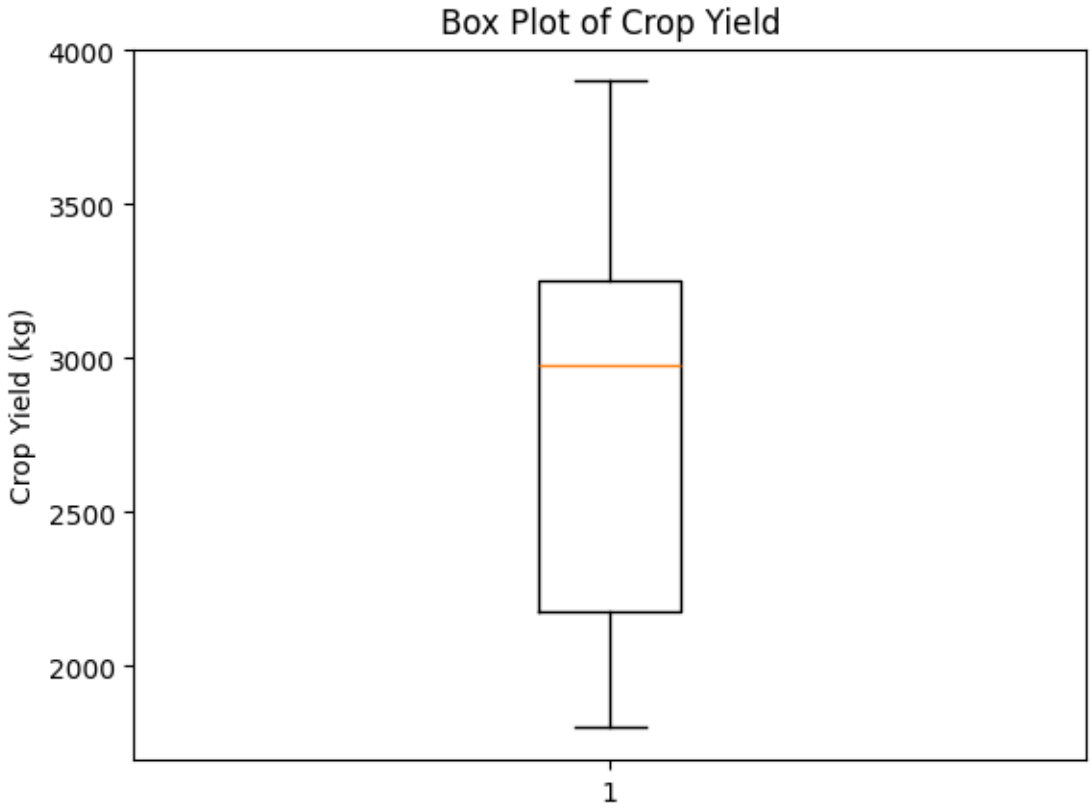
	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
Rainfall_mm	1.000000	-0.993192	0.980550	0.995521
Temperature	-0.993192	1.000000	-0.979735	-0.993300
Fertilizer_kg	0.980550	-0.979735	1.000000	0.983181
Crop_Yield_kg	0.995521	-0.993300	0.983181	1.000000

Average Crop Yield by Soil Type:
Soil_Type
Clay 2941.666667
Loamy 3514.285714
Sandy 2021.428571
Name: Crop_Yield_kg, dtype: float64


Average Crop Yield by Soil Type
Crop Yield (kg)
0
500
1000
1500
2000
2500
3000
3500
Clay
Loamy
Sandy
Soil Type


Crop Yield Distribution
Frequency
0
1
2
3
4
5
6
2000
2500
3000
3500
4000
Crop Yield (kg)





Observations:
1. Loamy soil has the highest average crop yield.
2. Higher rainfall generally results in higher crop yield.
3. Increased fertilizer usage is associated with higher crop production.

4. Lower temperatures with adequate rainfall produce better yields.
5. Sandy soil shows the lowest crop yield among all soil types.

Conclusion:
Rainfall, fertilizer usage, and soil type significantly influence crop yield. Loamy soil combined with adequate rainfall and fertilizer produces the highest agricultural output.